

- Modern framework tries to separate logic and UI as much as possible but, in Flutter, user interface and logic is intermixed. We can overcome this using smart coding and using high level module to separate user interface and logic.
- Flutter is yet another framework to create mobile application. Developers are having a hard time in choosing the right development tools in hugely populated segment.

Installation Flutter Framework

Flutter Installation in Windows

In this section, let us see how to install Flutter SDK and its requirement in a windows system.

Step 1: Go to URL, <https://flutter.dev/docs/get-started/install/windows> and download the latest Flutter SDK. As of April 2019, the version is 1.2.1 and the file is flutter_windows_v1.2.1-stable.zip.

Step 2: Unzip the zip archive in a folder, say C:\flutter\

Step 3: Update the system path to include flutter bin directory.

Step 4: Flutter provides a tool, flutter doctor to check that all the requirement of flutter development is met.

```
flutter doctor
```

Step 5: Running the above command will analyze the system and show its report as shown below:

```
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, v1.2.1, on Microsoft Windows [Version
10.0.17134.706], locale en-US)
[✓] Android toolchain - develop for Android devices (Android SDK version
28.0.3)
[✓] Android Studio (version 3.2)
[✓] VS Code, 64-bit edition (version 1.29.1)
[!] Connected device
    ! No devices available

! Doctor found issues in 1 category.
```

The report says that all development tools are available but the device is not connected.

We can fix this by connecting an android device through USB or starting an android emulator.

Step 6: Install the latest Android SDK, if reported by flutter doctor

Step 7: Install the latest Android Studio, if reported by flutter doctor

Step 8: Start an android emulator or connect a real android device to the system.

Step 9: Install Flutter and Dart plugin for Android Studio. It provides startup template to create new Flutter application, an option to run and debug Flutter application in the Android studio itself, etc.,

Open Android Studio.

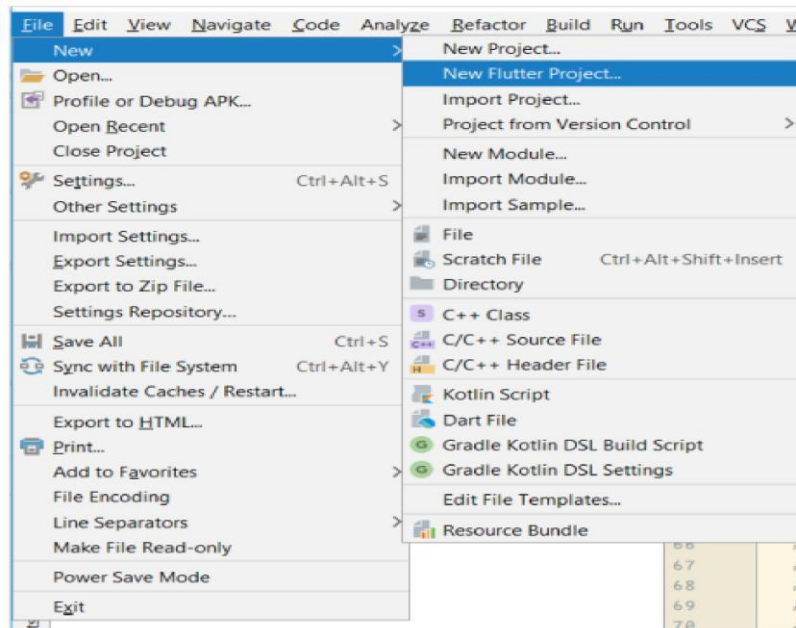
- Click File > Settings > Plugins.
- Select the Flutter plugin and click Install.
- Click Yes when prompted to install the Dart plugin.

- Restart Android studio.

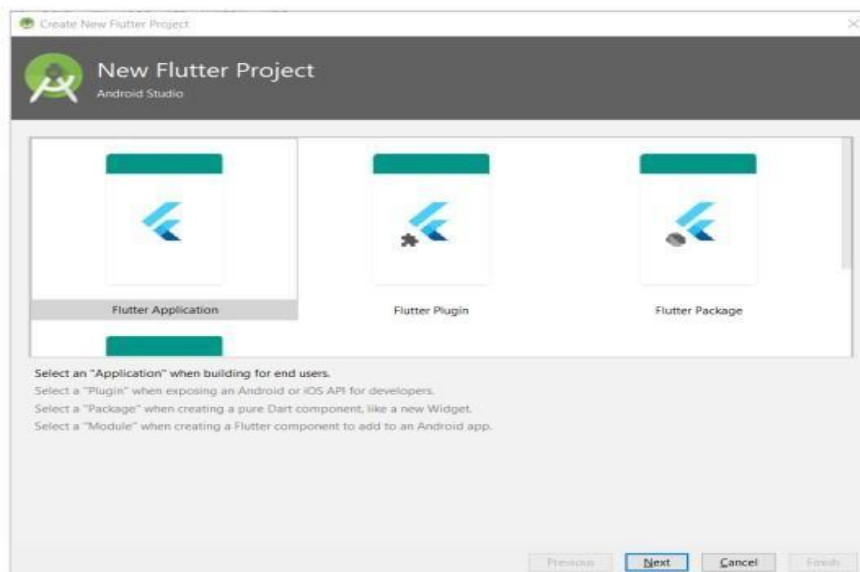
a simple *Flutter* application to understand the basics of creating a flutter application in the Android Studio.

Step 1: Open Android Studio

Step 2: Create Flutter Project. For this, click **File -> New -> New Flutter Project**.



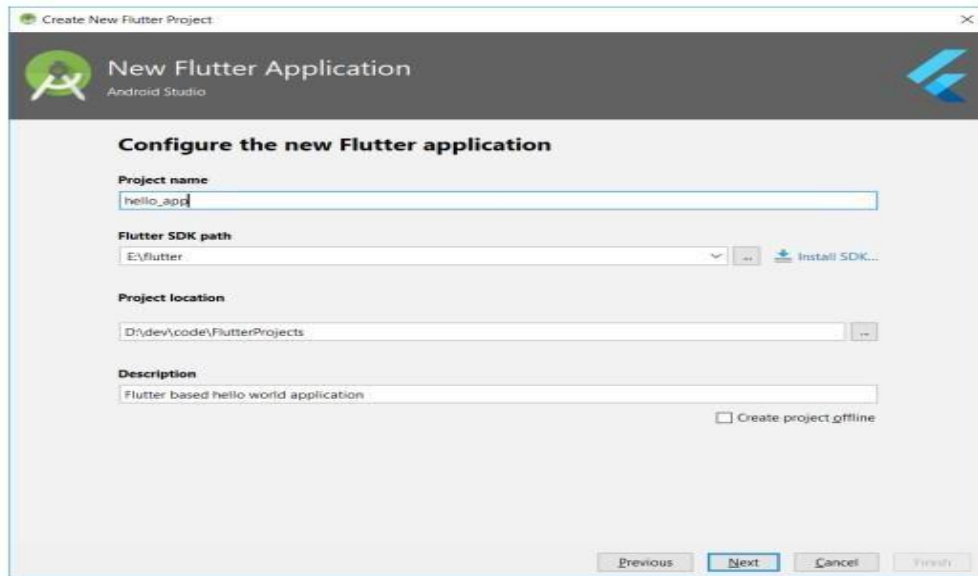
Step 3: Select Flutter Application. For this, select **Flutter Application** and click next.



Step 4: Configure the application as below and click **Next**.

- Project name: **hello_app**

- Flutter SDK Path: <path_to_flutter_sdk>
- Project Location: <path_to_project_folder>
- Description: Flutter based hello world application



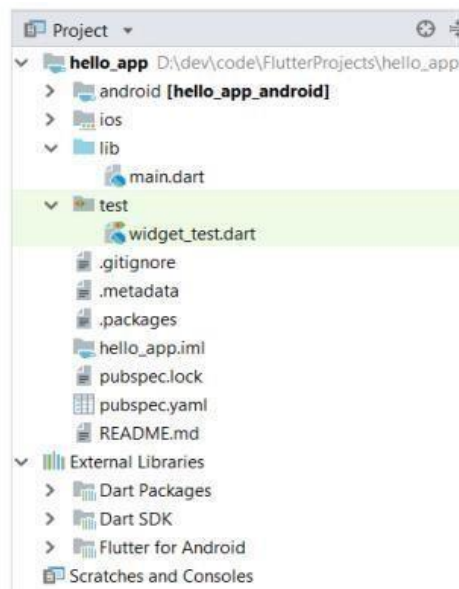
Step 5: Configure Project. Set the company domain as **flutterapp.tutorialspoint.com** and click **Finish**.

Step 6: Enter Company domain.

Android Studio creates a fully working flutter application with minimal functionality.

Let us check the structure of the application and then, change the code to do our task.

The structure of the application and its purpose is as follows:



Various components of the structure of the application are explained here:

- **android** - Auto generated source code to create android application
- **ios** - Auto generated source code to create ios application
- **lib** - Main folder containing Dart code written using flutter framework
- **lib/main.dart** - Entry point of the Flutter application
- **test** - Folder containing Dart code to test the flutter application
- **test/widget_test.dart** - Sample code
- **.gitignore** - Git version control file

- **.metadata** - auto generated by the flutter tools
- **.packages** - auto generated to track the flutter packages
- **.iml** - project file used by Android studio
- **pubspec.yaml** - Used by **Pub**, Flutter package manager
- **pubspec.lock** - Auto generated by the Flutter package manager, **Pub**
- **README.md** - Project description file written in Markdown format

Step 7: Replace the dart code in the lib/main.dart file with the below code:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Welcome to Flutter',
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Welcome to Flutter'),
        ),
        body: const Center(
          child: Text('Hello World'),
        ),
      ),
    );
  }
}
```

Let us understand the dart code line by line

1: imports the flutter package, material. The material is a flutter package to create user interface according to the Material design guidelines specified by Android.

3: This is the entry point of the Flutter application. Calls runApp function and pass it an object of MyApp class. The purpose of the runApp function is to attach the given widget to the screen.

4: Widget is used to create UI in flutter framework. StatelessWidget is a widget, which does not maintain any state of the widget. MyApp extends StatelessWidget and overrides its build method. The purpose of the build method is to create a part of the UI of the application. Here, build method uses MaterialApp, a widget to create the root level UI of the application. It has three properties - title, theme and home.

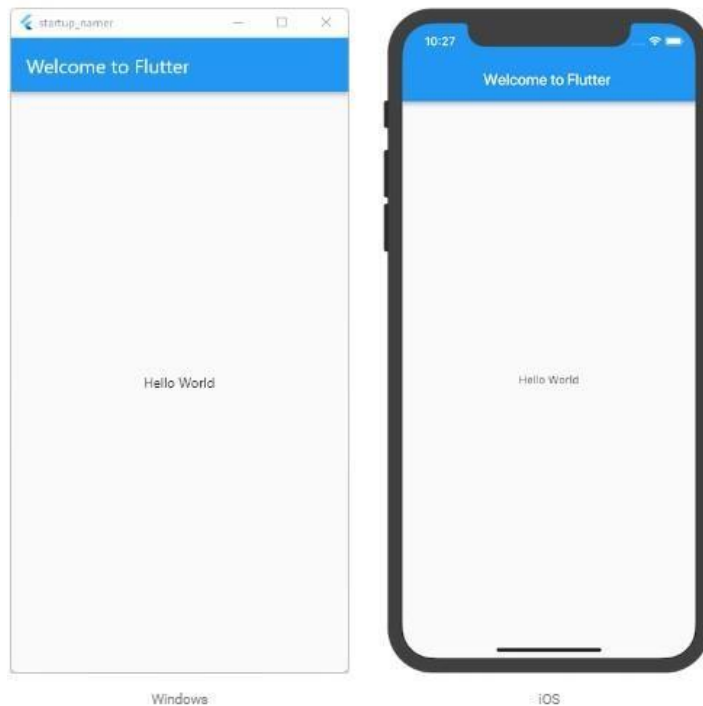
>- title is the title of the application.

>- theme is the theme of the widget. Here, we set blue as the overall color of the application using ThemeData class and its property, *primarySwatch*.

>- home is the inner UI of the application, which we set another widget, MyHomePage

5: MyHomePage is same as MyApp except it returns Scaffold Widget. Scaffold is a top level widget next to MaterialApp widget used to create UI conforming material design. It has two important properties, appBar to show the header of the application and body to show the actual content of the application. AppBar is another widget to render the header of the application and we have used it in appBar property. In body property, we have used Center widget, which centers its child widget. Text is the final and inner most widget to show the text and it is displayed in the center of the screen.

6: Now, run the application using, **Run -> Run main.dart** **7:** Finally, the output of the application is as follows:



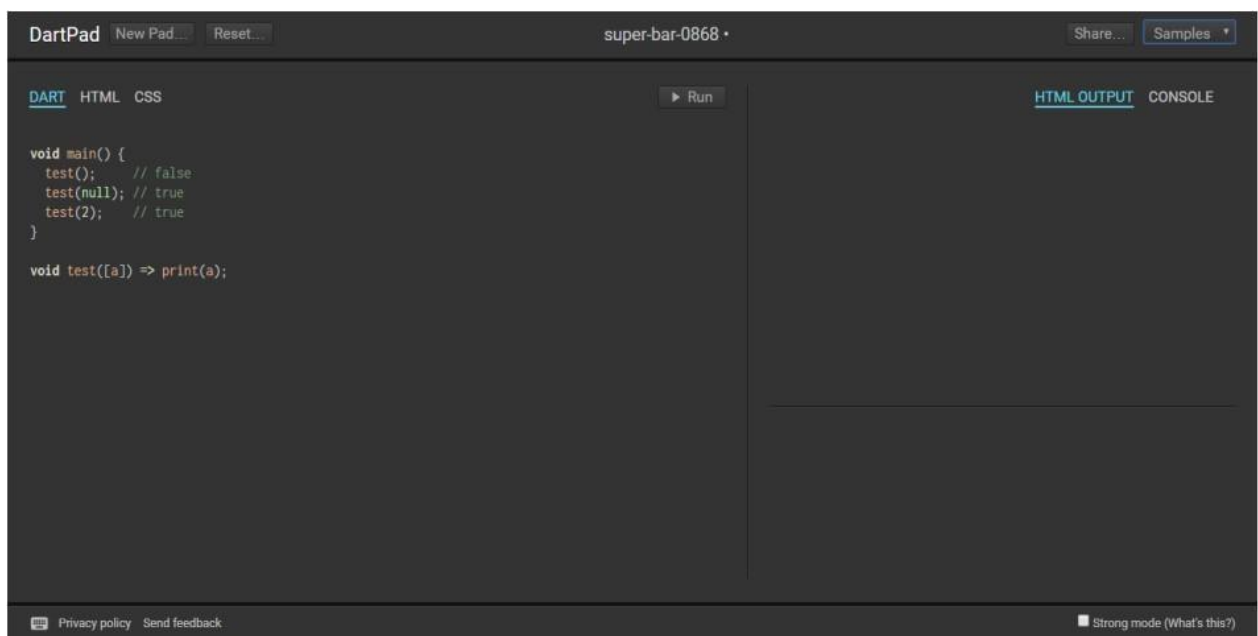
1.12 Dart programming language

Dart is an open-source general-purpose programming language. It is originally developed by Google and later approved as a standard by ECMA. Dart is a new programming language meant for the server as well as the browser. Introduced by Google, the Dart SDK ships with its compiler – the Dart VM. The SDK also includes a utility -dart2js, a transpiler that generates JavaScript equivalent of a Dart Script. Dart is an object-oriented language with Cstyle syntax which can optionally trans compile into JavaScript. It supports a varied range of programming aids like interfaces, classes, collections, generics, and optional typing. Dart can be extensively used to create single-page applications. Single-page applications apply only to websites and web applications. Single-page applications enable navigation between different screens of the website without loading a different webpage in the browser. A classic example is **GMail** – when you click on a message in your inbox, browser stays on the same webpage, but JavaScript code hides the inbox and brings the message body on screen. Google has released a special build of **Chromium** – the **Dart VM**.

Feature	Dart	JavaScript
Type system	Optional, dynamic	Weak, dynamic
Classes	Yes, single inheritance	Prototypical
Interfaces	Yes, multiple interfaces	No
Concurrency	Yes, with isolates	Yes, with HTML5 web workers

Dart Environment.

Executing Script Online with Dart Pad You may test your scripts online by using the online editor at <https://dartpad.dartlang.org/>. The Dart Editor executes the script and displays both HTML as well as console output. The online editor is shipped with a set of preset code samples. A screenshot of the **Dartpad** editor.



Dartpad also enables to code in a more restrictive fashion. This can be achieved by checking the Strong mode option on the bottom right of the editor. Strong mode helps with:

- Stronger static and dynamic checking
- Idiomatic JavaScript code generation for better interoperability.

You may try the following example using Dartpad

```
void main() {
  print('hello world');
}
```

The code will display the following output

```
hello world
```

Using the Text Editor

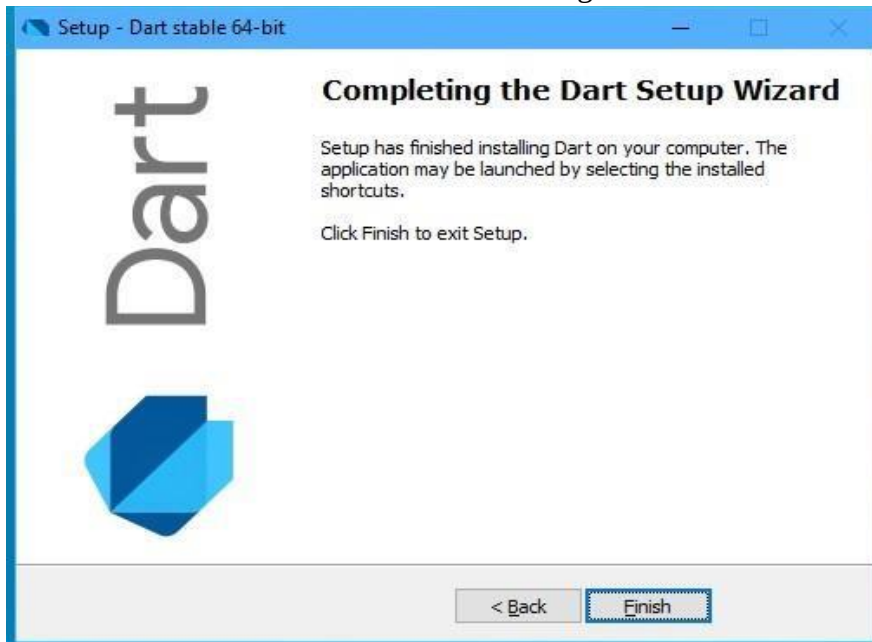
Examples of a few editors include Windows Notepad, Notepad++, Emacs, vim or vi, etc. Editors may vary from one Operating System to another. The source files are typically named with the extension ".dart".

Installing the Dart SDK

The current stable version of Dart is **1.21.0**. The **dart sdk** can be downloaded from:

- <https://www.dartlang.org/install/archive>
- <http://www.gekorm.com/dart-windows/>

A screenshot of the Dart SDK installation is given below:



On completion of the SDK installation, set the PATH environment variable to:

```
<dart-sdk-path>\bin
```

IDE Support Dart

A plethora of IDEs support scripting in Dart. Examples include Eclipse, IntelliJ, and WebStorm from Jet brains.

Given below are the steps for configuring the Dart environment using WebStorm IDE.

Installing WebStorm:

The installation file for WebStorm can be downloaded from

<http://www.jetbrains.com/webstorm/download/#section=windows-version>.

The WebStorm installation file is available for Mac OS, Windows and Linux.

After downloading the installation files, follow the steps given below:

- Install the Dart SDK: Refer to the steps listed above
- Create a new Dart project and configure Dart support • To create a new Dart project, o Click

- Create New Project from the Welcome Screen
 - o In the next dialog box, click Dart
- If there is no value specified for the Dart SDK path, then provide the SDK path. For example, the SDK path may be *<dart installation directory>/dart/dart-sdk*.

Add a Dart File to the Project:

To add a Dart file to the Project:

- Right-click on the Project
- New  Dart File
- Enter the name of the Dart Script The dart2js Tool:

The dart2js tool compiles Dart code to JavaScript. Compiling Dart code to JS enables running the Dart script on browsers that do not support the Dart VM. The dart2js tool is shipped as a part of the Dart SDK and can be found in the /dart sdk/bin folder.

To compile Dart to JavaScript, type the following command in the terminal

This command produces a file that contains the JavaScript equivalent of your Dart code. A

```
dart2js - - out=<output_file>.js <dart_script>.dart
```

Dart Syntax:

Syntax defines a set of rules for writing programs. Every language specification defines its own syntax. A Dart program is composed of:

- Variables and Operators
 - Classes
 - Functions
 - Expressions and Programming Constructs
 - Decision Making and Looping Constructs
 - Comments
 - Libraries and Packages
 - Typedefs
 - Data structures represented as Collections / Generics
- Dart Code**

```
main ()    {  
  
    Print ("Hello World")  
}
```

The **main ()** function is a predefined method in Dart. This method acts as the entry point to the application. A Dart script needs the **main ()** method for execution. **print ()** is a predefined function that prints the specified string or value to the standard output i.e., the terminal. The output of the above code will be:

```
Hello World!
```

Execute a Dart Program

You can execute a Dart program in two ways:

- Via the terminal
- Via the WebStorm IDE Via the Terminal To execute a Dart program via the terminal:
- Navigate to the path of the current project
- Type the following command in the Terminal window

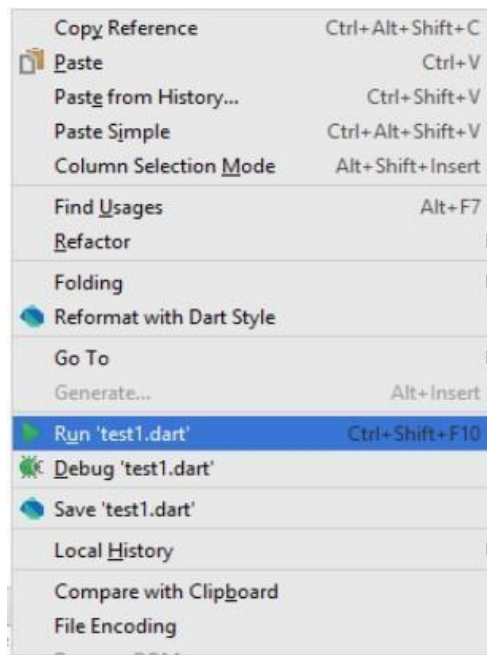
```
dart file_name. Dart
```

Via the WebStorm IDE

To execute a Dart program via the WebStorm IDE:

- Right-click the Dart script file on the IDE. (The file should contain the main () function to enable execution)
- Click on the 'Run <file_name>' option.

One can alternatively click the button Run or use the shortcut **Ctrl+Shift+F10** to execute the Dart Script. 'test1.dart'

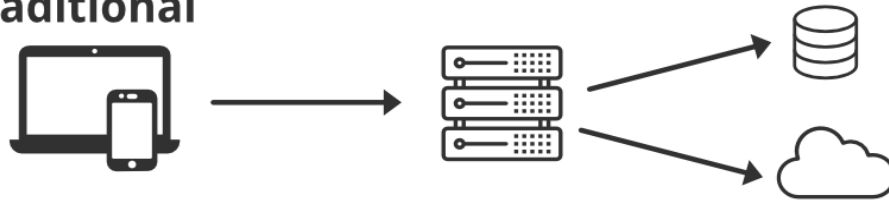


1.13 Firebase

Firestore is a Backend-as-a-Service (BaaS) app development platform that provides hosted backend services such as a realtime database, cloud storage, authentication, crash reporting, machine learning, remote configuration, and hosting for your static files.

Firestore is a product of Google which helps developers to build, manage, and grow their apps easily. It helps developers to build their apps faster and in a more secure way. No programming is required on the firestore side which makes it easy to use its features more efficiently. It provides services to android, ios, web, and unity. It provides cloud storage. It uses NoSQL for the database for the storage of data.

Traditional



Firebase

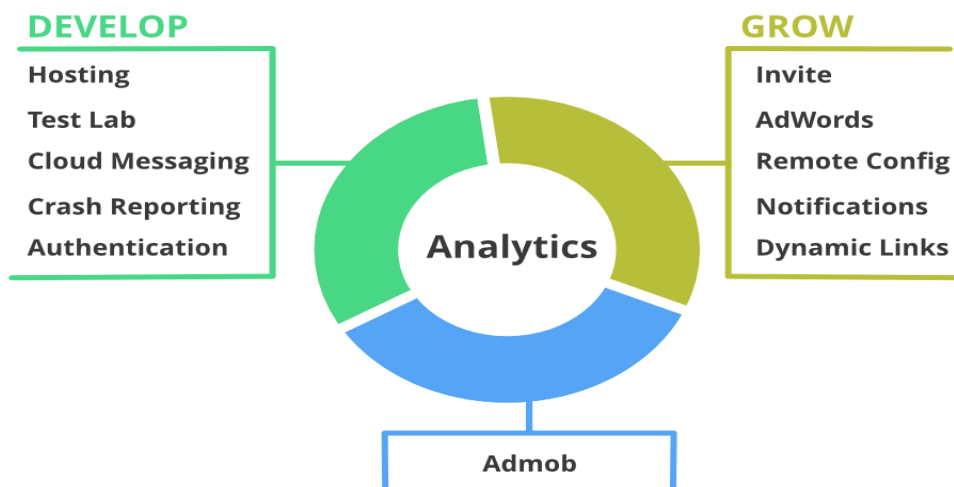


Brief History of Firebase:

Firebase initially was an online chat service provider to various websites through API and ran with the name **Envolve**. It got popular as developers used it to exchange application data like a game state in real time across their users more than the chats. This resulted in the separation of the Envolve architecture and its chat system. The Envolve architecture was further evolved by its founders James Tamplin and Andrew Lee, to what modern day Firebase is in the year 2012.

Features of Firebase:

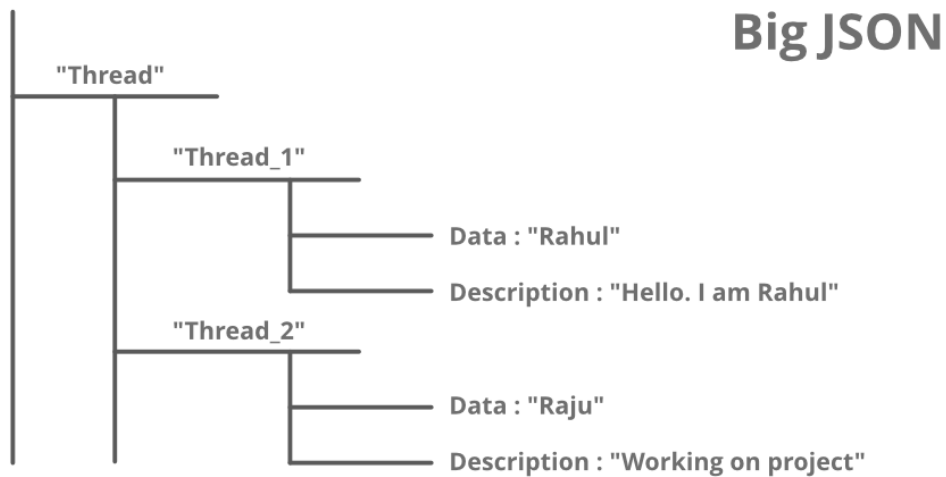
Mainly there are 3 categories in which firebase provides its services.



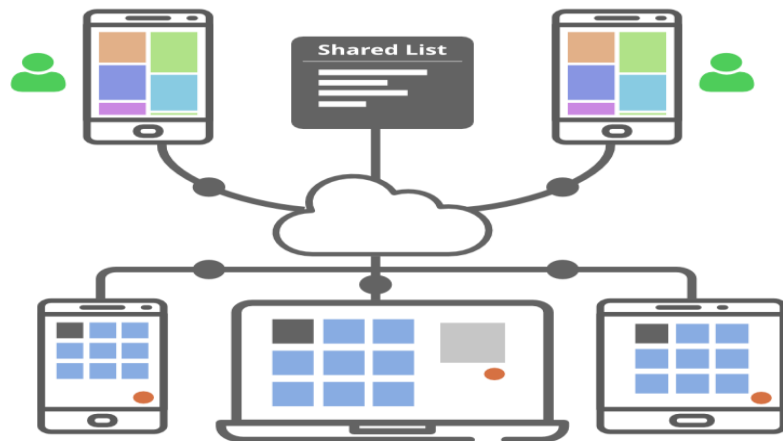
Build better applications

This feature mainly includes backend services that help developers to build and manage their applications in a better way. Services included under this feature are :

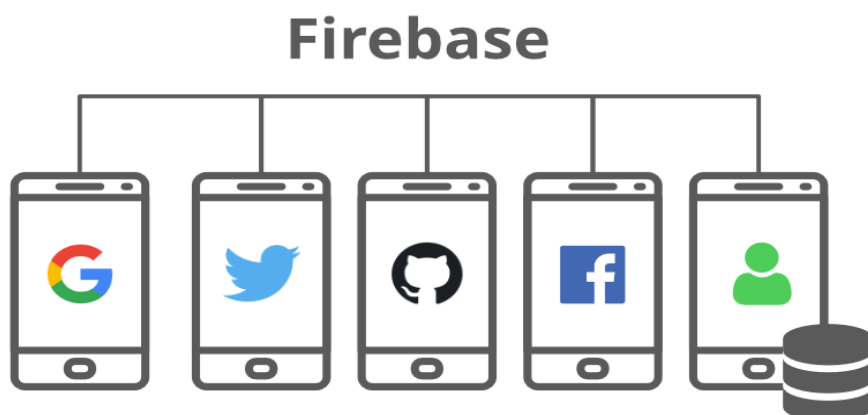
Realtime Database: The Firebase Realtime Database is a cloud-based NoSQL database that manages your data at the blazing speed of milliseconds. In simplest term, it can be considered as a big JSON file



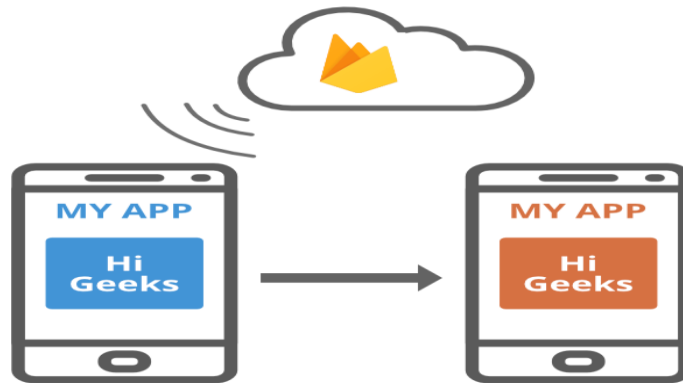
Cloud Firestore: The cloud Firestore is a NoSQL document database that provides services like store, sync, and query through the application on a global scale. It stores data in the form of objects also known as Documents. It has a key-value pair and can store all kinds of data like, strings, binary data, and even JSON trees.



Authentication: Firebase Authentication service provides easy to use UI libraries and SDKs to authenticate users to your app. It reduces the manpower and effort required to develop and maintain the user authentication service. It even handles tasks like merging accounts, which if done manually can be hectic.

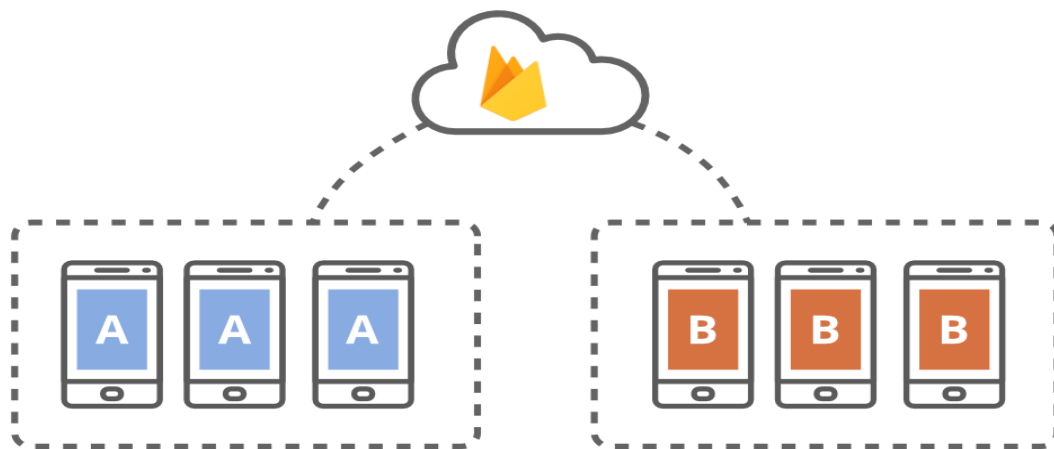


Remote Config: The remote configuration service helps in publishing updates to the user immediately. The changes can range from changing components of the UI to changing the behavior of the applications. These are often used while publishing seasonal offers and contents to the application that has a limited life.



Hosting: Firebase provides hosting of applications with speed and security. It can be used to host Static or Dynamic websites and microservices. It has the capability of hosting an application with a single command.

Firebase Cloud Messaging(FCM): The FCM service provides a connection between the server and the application end users, which can be used to receive and send messages and notifications. These connections are reliable and battery-efficient.



Improve app quality:

Here majorly all the application performance and testing features are provided. All the features required to check and manage before launching your application officially are provided in this section. Services included are:

- **Crashlytics:** It is used to get real-time crash reports. These reports can further be used to improve the quality of the application. The most interesting part of this service is that it gives a detailed description of the crash which is easier to analyze for the developers.
- **Performance monitoring:** This service gives an insight to the performance characteristics of the applications. The performance monitoring SDK can be used to receive performance data from the application, review them, and make changes to the application accordingly through the Firebase console.

- **Test lab:** This service helps to test your applications on real and virtual devices provided by Google which are hosted on the Google Datacenters. It is a cloud-based app-testing infrastructure which supports testing the application on a wide variety of devices and device configurations
- **App Distribution:** This service is used to pre-release applications that can be tested by trusted testers. It comes in handy as it decreases the time required to receive feedback from the testers.

Grow your app:

This feature provides your application analytics and features that can help you to interact with your user and make predictions that help you to grow your app. Services provided are:

- **Google analytics:** It is a Free app measurement service provided by Google that provides insight on app usage and user engagement. It serves unlimited reporting for up to 500 distinct automatic or user-defined events using the Firebase SDK.
- **Predictions:** Firebase Predictions uses machine learning to the application's analytics data, further creating dynamic user segments that are based on your user's behavior. These are automatically available to use for the application through Firebase Remote Config, the Notifications composer, Firebase In-App Messaging, and A/B Testing.
- **Dynamic Links:** Deep Links are links that directly redirect user to specific content. Firebase provides a Dynamic linking service that converts deep links into dynamic links which can directly take the user to a specified content inside the application. Dynamic links are used for converting web users to Native app users. It also increases the conversion of user-to-user sharing. In addition, it can also be used to integrate social media networks, emails, and SMS to increase user engagement inside the application.
- **A/B Testing:** It is used to optimize the application's experience by making it run smoothly, scaling the product, and performing marketing experiments.

Pros and Cons of Using Firebase:

Below listed are the advantages and disadvantages of using a Firebase backend:

Pros:

- Free plans for beginners.
- Real-time database is available.
- Growing Community.
- Numerous services are available.

Cons:

- It uses NoSQL so, people migrating from SQL might feel difficulty.
- It is still growing so, it is not tested to an extent.

Companies using Firebase

Below are some reputable organizations that rely on a firebase backend for its functioning:

- The New York Times
- Alibaba.com
- Gameloft
- Duolingo
- Trivago
- Venmo
- Lyft

Pricing:

There are 2 plans available. Spark plan is initially free but as your user base grows you might need to upgrade to blaze plan. Firebase asks you to pay as you go. For most developers who are just getting started and are on a learning path, they are covered by spark plan.

1.14 Add Firebase to your Flutter app

Prerequisites

- Install your preferred [editor or IDE](#).
- Set up a physical Apple device or use a simulator to run your app.
- Make sure that your Flutter app targets the following platform versions or later:
 - iOS 11
 - macOS 10.13
- [Install Flutter](#) for your specific operating system, including the following:
 - Flutter SDK
 - Supporting libraries
 - Platform-specific software and SDKs
- [Sign into Firebase](#) using your Google account.

If you don't already have a Flutter app, you can complete the [Get Started: Test Drive](#) to create a new Flutter app using your preferred editor or IDE.

Step 1: Install the required command line tools

1. If you haven't already, [install the Firebase CLI](#).
2. Log into Firebase using your Google account by running the following command:
firebase login
3. Install the FlutterFire CLI by running the following command from any directory:
dart pub global activate flutterfire_cli

Step 2: Configure your apps to use Firebase

Use the FlutterFire CLI to configure your Flutter apps to connect to Firebase.

From your Flutter project directory, run the following command to start the app configuration workflow:

flutterfire configure

After this initial running of **flutterfire configure**, you need to re-run the command any time that you:

- Start supporting a new platform in your Flutter app.
- Start using a new Firebase service or product in your Flutter app, especially if you start using sign-in with Google, Crashlytics, Performance Monitoring, or Realtime Database.

Re-running the command ensures that your Flutter app's Firebase configuration is up-to-date and (for Android) automatically adds any required Gradle plugins to your app.

Step 3: Initialize Firebase in your app

1. From your Flutter project directory, run the following command to install the core plugin:
flutter pub add firebase_core
2. From your Flutter project directory, run the following command to ensure that your Flutter app's Firebase configuration is up-to-date:
flutterfire configure
3. In your lib/main.dart file, import the Firebase core plugin and the configuration file you generated earlier:
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

4. Also in your lib/main.dart file, initialize Firebase using the DefaultFirebaseOptions object exported by the configuration file:
**await Firebase.initializeApp(
 options: DefaultFirebaseOptions.currentPlatform,
);**
5. Rebuild your Flutter application:
flutter run

Step 4: Add Firebase plugins

You access Firebase in your Flutter app through the various [Firebase Flutter plugins](#), one for each Firebase product (for example: Cloud Firestore, Authentication, Analytics, etc.).

Since Flutter is a multi-platform framework, each Firebase plugin is applicable for Apple, Android, and web platforms. So, if you add any Firebase plugin to your Flutter app, it will be used by the Apple, Android, and web versions of your app.

Here's how to add a Firebase Flutter plugin:

1. From your Flutter project directory, run the following command:
flutter pub add *PLUGIN_NAME*
2. From your Flutter project directory, run the following command:
flutterfire configure

Running this command ensures that your Flutter app's Firebase configuration is up-to-date and, for Crashlytics and Performance Monitoring on Android, adds the required Gradle plugins to your app.

3. Once complete, rebuild your Flutter project:
flutter run

You're all set! Your Flutter apps are registered and configured to use Firebase.

1.15 Version Control

How version control helps high performing development and DevOps teams prosper Version control, also known as source control, is the practice of tracking and managing changes to software code. Version control systems are software tools that help software teams manage changes to source code over time. As development environments have accelerated, version control systems help software teams work faster and smarter. They are especially useful for DevOps (<https://www.atlassian.com/devops/what-is-devops>) teams since they help them to reduce development time and increase successful deployments. Version control software keeps track of every modification to the code in a special kind of database. If a mistake is made, developers can turn back the clock and compare earlier versions of the code to help fix the mistake while minimizing disruption to all team members.

For almost all software projects, the source code is like the crown jewels - a precious asset whose value must be protected. For most software teams, the source code is a repository of the invaluable knowledge and understanding about the problem domain that the developers have collected and refined through careful effort. Version control protects source code from both catastrophe and the casual degradation of human error and unintended consequences.

Software developers working in teams are continually writing new source code and changing existing source code. The code for a project, app or software component is typically organized in a folder structure or "file tree". One developer on the team may be working on a new feature while another developer fixes an unrelated bug by changing code, each developer may make their changes in several parts of the file tree.

Version control helps teams solve these kinds of problems, tracking every individual change by each contributor and helping prevent concurrent work from conflicting. Changes made in one part of the software can be incompatible with those made by another developer working at the same time. This problem should be discovered and solved in an orderly manner without blocking the work of the rest of the team. Further, in all software development, any change can introduce new bugs on its own and new software can't be trusted until it's tested. So, testing and development proceed together until a new version is ready. Good version control software supports a developer's preferred workflow without imposing one particular way of working. Ideally it also works on any platform, rather than dictate what operating system or tool chain developers must use. Great version control systems facilitate a smooth and continuous flow of changes to the code rather than the frustrating and clumsy mechanism of file locking - giving the green light to one developer at the expense of blocking the progress of others. Software teams that do not use any form of version control often run into problems like not knowing which changes that have been made are available to users or the creation of incompatible changes between two unrelated pieces of work that must then be painstakingly untangled and reworked. If you're a developer who has never used version control you may have added versions to your files, perhaps with suffixes like "final" or "latest" and then had to later deal with a new final version. Perhaps you've commented out code blocks because you want to disable certain functionality without deleting the code, fearing that there may be a use for it later. Version control is a way out of these problems. Version control software is an essential part of the every-day of the modern software team's professional practices. Individual software developers who are accustomed to working with a capable version control system in their teams typically recognize the incredible value version control also gives them even on small solo projects. Once accustomed to the powerful benefits of version control systems, many developers wouldn't consider working without it even for non-software projects.



1.16 Benefits of version control systems

Using version control software is a best practice for high performing software and DevOps (<https://www.atlassian.com/devops/what-is-devops>) teams. Version control also helps developers

move faster and allows software teams to preserve efficiency and agility as the team scales to include more developers.

Version Control Systems (VCS) have seen great improvements over the past few decades and some are better than others. VCS are sometimes known as SCM (Source Code Management) tools or RCS (Revision Control System). One of the most popular VCS tools in use today is called Git. Git is a Distributed VCS, a category known as DVCS, more on that later. Like many of the most popular VCS systems available today, Git is free and open source. Regardless of what they are called, or which system is used, the primary benefits you should expect from version control are as follows.

- 1- A complete long-term change history of every file. This means every change made by many individuals over the years. Changes include the creation and deletion of files as well as edits to their contents. Different VCS tools differ on how well they handle renaming and moving of files. This history should also include the author, date and written notes on the purpose of each change. Having the complete history enables going back to previous versions to help in root cause analysis for bugs and it is crucial when needing to fix problems in older versions of software. If the software is being actively worked on, almost everything can be considered an "older version" of the software.
- 2- Branching and merging. Having team members work concurrently is a no-brainer, but even individuals working on their own can benefit from the ability to work on independent streams of changes. Creating a "branch" in VCS tools keeps multiple streams of work independent from each other while also providing the facility to merge that work back together, enabling developers to verify that the changes on each branch do not conflict. Many software teams adopt a practice of branching for each feature or perhaps branching for each release, or both. There are many different workflows that teams can choose from when they decide how to make use of branching and merging facilities in VCS.
- 3- Traceability. Being able to trace each change made to the software and connect it to project management and bug tracking software such as Jira (<https://www.atlassian.com/software/jira>), and being able to annotate each change with a message describing the purpose and intent of the change can help not only with root cause analysis and other forensics. Having the annotated history of the code at your fingertips when you are reading the code, trying to understand what it is doing and why it is so designed can enable developers to make correct and harmonious changes that are in accord with the intended long-term design of the system. This can be especially important for working effectively with legacy code and is crucial in enabling developers to estimate future work with any accuracy.

While it is possible to develop software without using any version control, doing so subjects the project to a huge risk that no professional team would be advised to accept. So the question is not whether to use version control but which version control system to use.

1.17 Why do I need version control software?

VCS is an invaluable tool with many benefits to a collaborative software team workflow. Any software project that has more than one developer maintaining source code files should absolutely use a VCS. Additionally, sole-maintainer projects will also greatly benefit from utilizing a VCS. There is arguably no valid reason to forgo the use of a VCS in any modern software development project.